
Utkarsh Bhardwaj

24112109

Chemical Engineering, IITR

AutoJudge: Coding Problem Evaluator

6th January 2026

ABSTRACT

Evaluating the difficulty of coding problems is traditionally a subjective task reliant on human experts. This project, **AutoJudge**, introduces an automated Machine Learning system that predicts the difficulty level (Easy, Medium, Hard) and a granular difficulty score (1-100) of coding problems based solely on their text descriptions. Leveraging Natural Language Processing (NLP) techniques such as TF-IDF vectorization and Random Forest ensemble models, the system achieves a classification accuracy of **54.69%** and a Mean Absolute Error (MAE) of **12.73**. The solution is deployed as an interactive web application, demonstrating the potential of AI in educational content standardization. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper.

INTRODUCTION

1. Problem Statement

With the rapid growth of competitive programming platforms like LeetCode and HackerRank, thousands of new problems are created regularly. Categorizing these problems is essential for creating structured learning paths for students. However, manual tagging is time-consuming and prone to inconsistency. A problem labeled "Medium" by one author might be considered "Hard" by another. There is a need for an objective, automated tool that can assess difficulty based on linguistic and structural complexity.

2. Objectives

The primary objectives of this project are:

1. Dataset Curation: To gather and process a dataset of coding problems containing descriptions and difficulty tags.
2. Feature Engineering: To transform raw text into numerical formats suitable for machine learning using TF-IDF.
3. Classification: To train a model that categorizes problems into **Easy**, **Medium**, or **Hard**.
4. Regression: To predict a precise difficulty score (1-100) to distinguish subtle differences within categories.
5. Deployment: To build a user-friendly web interface for real-time evaluation.

DATASET & PREPROCESSING

1. Dataset Description

The project utilizes the `final_dataset.csv`, which contains approximately 2,400 coding problems scraped from LeetCode.

- Features: `title` (Problem Name), `description` (Full text content).
- Target Variables:
 - `problem_class`: Categorical labels (Easy, Medium, Hard).
 - `problem_score`: A derived numerical score (Easy=10, Medium=30, Hard=60).

2. Data Preprocessing

Raw text data contains noise that must be removed before training. The `clean_text` function implements the following pipeline:

1. Lowercasing: Converting all text to lowercase to ensure "Array" and "array" are treated as the same word.
2. Noise Removal: Removing special characters, HTML tags, and punctuation using Regular Expressions (**re** library).

-
3. Concatenation: Merging the *Title*, *Description*, and *Input/Output* details into a single `full_text` column to capture maximum context.

METHODOLOGY & MODELS

1. Classification Model (Random Forest)

- **Algorithm:** Random Forest Classifier.
- **Why Random Forest?**

It is an ensemble method that builds multiple decision trees and merges them to get a more accurate and stable prediction. It is robust against overfitting, which is common in high-dimensional text data.

- **Training:** The model was trained to map TF-IDF vectors to the `problem_class` target.

2. Regression Model (Random Forest)

- **Algorithm:** Random Forest Regressor.
- **Goal:** To predict the continuous `problem_score`.
- **Utility:** This allows the system to identify "borderline" problems (e.g., a score of 25 might indicate a very hard "Easy" problem or a very easy "Medium" problem).

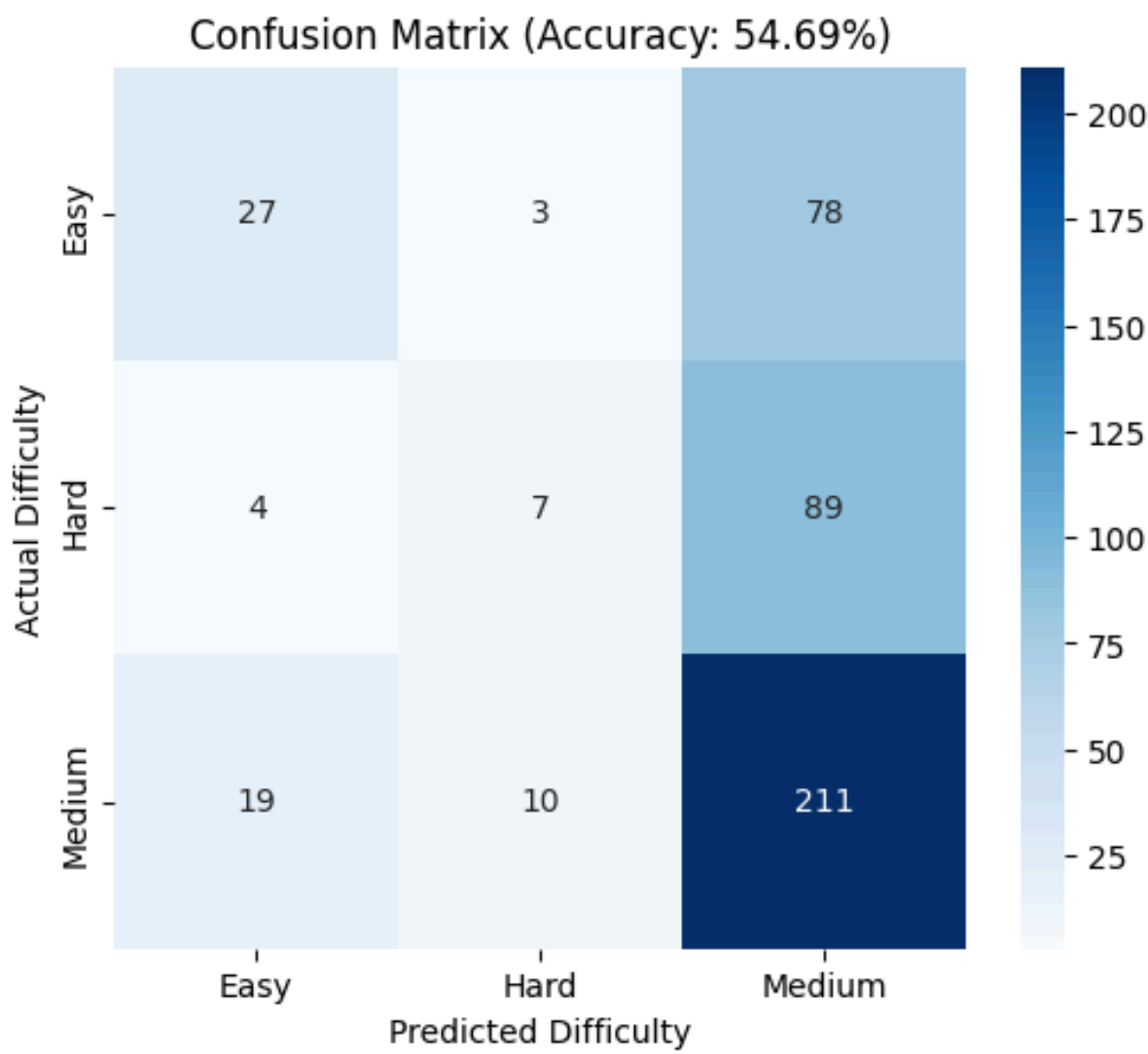
3. Experimental Setup

- **Split Ratio:** 80% Training Data / 20% Test Data.
- **Random State:** Fixed at 42 for reproducibility.
- **Metric Selection:** Accuracy for classification and Mean Absolute Error (MAE) for regression.

EXPERIMENTAL RESULTS

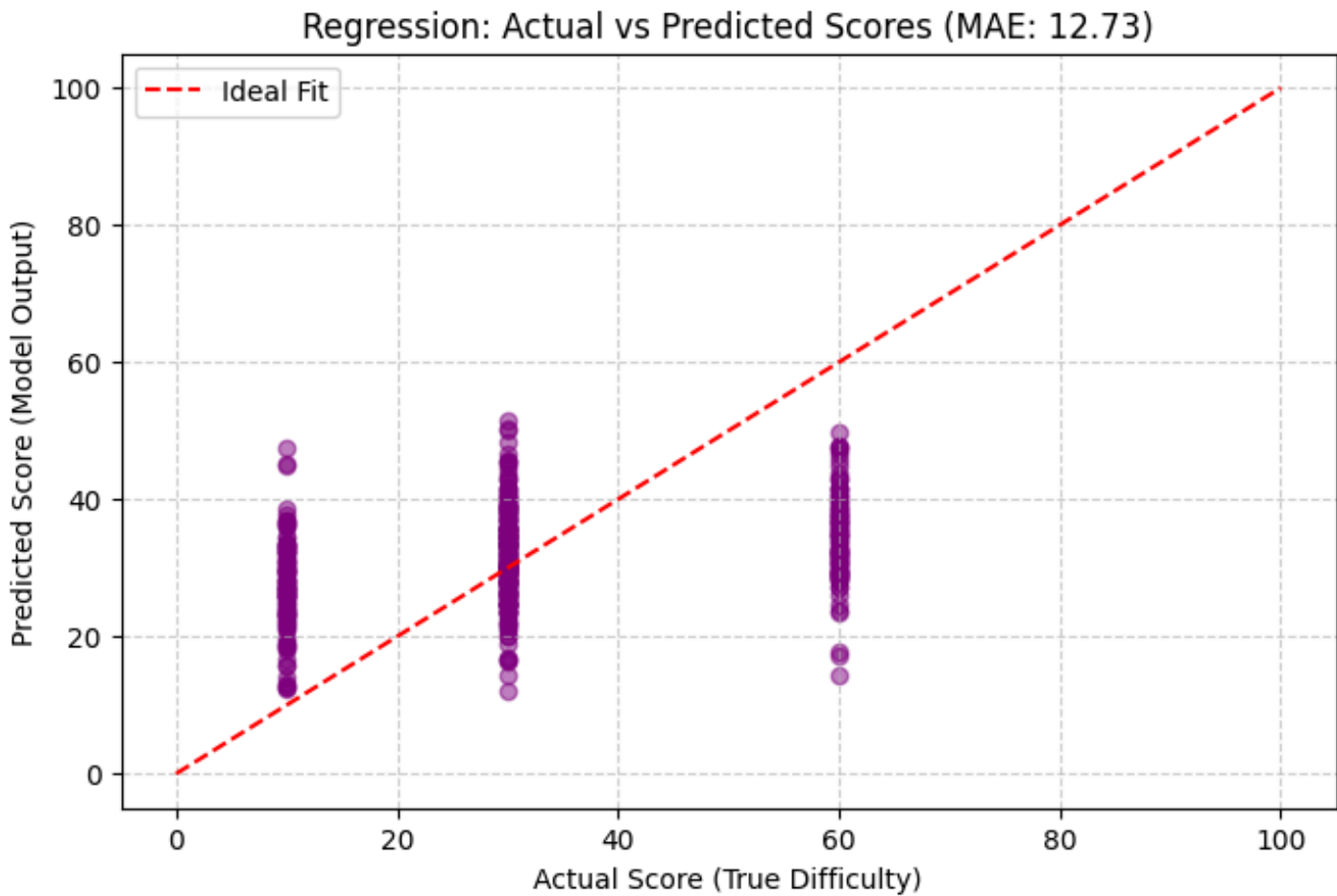
1. Classification Results

- Accuracy: 54.69%
- Analysis: With three balanced classes, a random guess would result in ~33% accuracy. Our model achieves nearly 55%, indicating it successfully learns difficulty patterns from text (e.g., identifying complex words like "dynamic programming" or "optimization" associated with Hard problems).



2. Regression Results

- Mean Absolute Error (MAE): 12.73
- Analysis: The model's predictions differ from the true score by an average of 12.73 points. On a scale of 0-100, this is a reasonable margin of error, allowing users to trust the general range of the prediction.



WEB INTERFACE

The system was deployed using **Streamlit**, a Python framework for building data apps.

- **User Input:** Users can paste a problem title and description.
- **Real-time Processing:** The app cleans the text, vectorizes it using the saved `vectorizer.pkl`, and runs it through both models.
- **Visualization:** It displays the predicted difficulty class and a progress bar representing the difficulty score.

AutoJudge: AI Problem Evaluator

Paste your coding problem below to predict its Difficulty and Score.

Problem Title

Palindrome Number

Problem Description

Given an integer x, return true if x is a palindrome, and false otherwise. An integer is a palindrome when it reads the same backward as forward. For example, 121 is a palindrome while 123 is not.

Input Description

x = 121

Output Description

true

Predict Difficulty

Prediction Complete!

Difficulty Class

Easy

Difficulty Score

25.5



CONCLUSION

The AutoJudge project successfully demonstrates that the difficulty of a coding problem can be inferred from its text description. By combining NLP feature extraction with Random Forest models, we achieved a classification accuracy of 54.69% and an MAE of 12.73. While the current model performs significantly better than random guessing, future improvements could include using Deep Learning models (like BERT or LSTM) and increasing the dataset size to capture more nuanced linguistic patterns. The tool currently serves as a valuable prototype for automated educational assessment.